

8 프로세스 별 추가 (복구 시스템)

핵심만 말하면:

- 1~6 프로세스는 그대로 유지
- 대신 각 프로세스에 복구 상태/복구 이벤트 처리를 추가
- 특히 **Host Simulator, Local Agent, Monitor, Controller/UI** 쪽 보강이 크고
- **R1, R2는 거의 그대로**, 이벤트 타입만 하나 더 늘어난다고 보면 된다.

아래처럼 정리하면 된다.

1. Host Simulator에 추가할 것

Host Simulator는 원래도 장애 발생 + 복구 상태 전환까지 담당하는 구조로 잡혀 있다. 복구를 명시 이벤트로 쓰려면 여기서 더 분명해져야 한다.

추가/명확화할 내용:

- 각 장애에 대해 **정상 상태 기준값**을 명확히 둬
- `fault_mode` 외에 **recovery 가능한 상태 전이** 규칙을 둬
- duration 만료 또는 수동 명령 시 **정상 상태로 복귀**
- 복귀가 일어났다는 사실이 Agent가 감지 가능하도록 상태가 확실히 바뀌어야 함

실질적으로 추가할 상태:

- `previous_fault_mode`
- `recovery_pending` 또는 명시적 정상 복귀 시점
- 장애별 정상 범위 정의
 - CPU 정상 범위
 - service 정상값 = `UP`
 - latency 정상값 = `NORMAL`

즉 Host Simulator는 “장애를 끝내는 것”만 하지 말고, 복구가 감지 가능한 상태 전이를 제공해야 한다.

2. Local Agent에 추가할 것

여기가 가장 중요하다.

이미 정리한 구조에도 recovery 이벤트 항목이 들어가 있다. `CPU_RECOVERED`, `SERVICE_RECOVERED`, `LATENCY_NORMALIZED` 같은 이벤트를 **비정상 → 정상 전이 시** 생성하도록 되어 있다.

추가/명확화할 내용:

- 장애 감지 규칙뿐 아니라 **복구 감지 규칙**을 각 항목별로 둬
- **직전 상태**(`last_host_status`) 와 현재 상태 비교 시

- 정상 → 비정상 = 장애 이벤트
- 비정상 → 정상 = 복구 이벤트
- 복구 이벤트도 일반 이벤트와 동일하게
 - `event_id`
 - `seq_no`
 - `timestamp`
 - `payload`

를 부여해서 publish

구체적으로 추가할 이벤트:

- `CPU_RECOVERED`
- `MEMORY_RECOVERED`
- `SERVICE_RECOVERED`
- `LATENCY_NORMALIZED`

추가할 내부 규칙:

- 정상 상태 polling 때마다 recovery를 계속 만들면 안 됨
- 반드시 **상태 전이 기반**이어야 함

예: `DOWN → UP` 일 때만 `SERVICE_RECOVERED`

즉 Agent는 이제 “장애 검출기”에서

장애/복구 전이 검출기로 완성된다.

3. Relay R1에 추가할 것

R1은 구조적으로 크게 바뀌지 않는다.

왜냐하면 R1 입장에서는:

- `CPU_SPIKE`
- `SERVICE_DOWN`
- `SERVICE_RECOVERED`

모두 그냥 **EVENT** 타입 문자열이 다른 동일한 메시지이기 때문이다.

추가/명확화할 내용:

- 복구 이벤트도 일반 이벤트처럼
 - validate
 - dedup
 - ACK
 - retry
 - forward
- 처리

- 로그에 `event_type` 이 recovery인지 fault인지 정도는 표시

즉 R1에서 추가되는 건 사실상:

- 새 `event_type` 허용
- 로그/통계 분리 정도

구조 변화는 거의 없다.

4. Relay R2에 추가할 것

R2도 R1과 동일하다.

추가/명확화할 내용:

- recovery 이벤트도 일반 이벤트와 똑같이 처리
- 후단 전달 실패 시 recovery 이벤트도 fault 이벤트와 같은 수준으로 retry/buffer/failover 적용
- Monitor로 넘길 때 `event_type` 유지

주의할 점:

- 복구 이벤트를 더 낮은 우선순위로 두면 안 된다
- 장애 이벤트는 갔는데 복구 이벤트가 안 가면 Monitor가 영구 장애 상태처럼 남을 수 있음

즉 R2에서는 복구 이벤트도 동일 신뢰성 수준으로 전달해야 한다.

5. Monitor에 추가할 것

여기가 두 번째로 중요하다.

Monitor는 원래 최종 수신, 로그 저장, 현재 상태 갱신을 담당한다. 복구 이벤트를 넣으려면 **현재 상태 테이블 갱신 규칙**이 더 분명해져야 한다.

추가/명확화할 내용:

- 장애 이벤트와 복구 이벤트를 **딱으로 해석**하는 규칙 추가
- `host_state_table` 에 현재 상태뿐 아니라 **최근 장애/복구 이력** 반영
- recent events에 recovery도 표시
- 필요하면 장애 지속 시간 계산 가능하게
 - 장애 시작 시각
 - 복구 시작를 기록

상태 갱신 규칙 예:

- `CPU_SPIKE` → `cpu_state = HIGH`
- `CPU_RECOVERED` → `cpu_state = NORMAL`
- `SERVICE_DOWN` → `service_state = DOWN`
- `SERVICE_RECOVERED` → `service_state = UP`

추가하면 좋은 것:

- `active_issues` 개념
 - 장애 이벤트 오면 active 추가
 - 복구 이벤트 오면 active 제거
- `last_recovery_time`

즉 Monitor는 이제 단순히 "장애가 왔다"를 넘어서

현재 장애가 아직 진행 중인지, 복구됐는지를 명확히 보여줄 수 있어야 한다.

6. Controller / UI에 추가할 것

이쪽도 이미 `Recovery` 버튼과 `recover` 명령 개념이 들어가 있다. 즉 구조는 맞다. 다만 복구 이벤트를 명시적으로 쓰기로 했으니 UI 해석도 그에 맞춰 더 분명해져야 한다.

추가/명확화할 내용:

- Host 장애 주입 영역에 각 장애별 복구 버튼 명확화
 - Recover CPU
 - Recover Service
 - Normalize Latency
- 최근 이벤트 영역에 recovery 이벤트도 표시
- 상태 표시 영역에서
 - 장애 상태
 - 복구 상태
 - 현재 정상 상태
 를 구분해서 보이게 함
- 필요하면 **active issue count** 표시
- 명령 이력에 recovery 명령도 남김

UI 동작 예:

- `CPU Spike` 클릭 → Host fault 주입
- `Recover CPU` 클릭 → Host 정상 복귀
- Agent가 `CPU_RECOVERED` 생성
- UI recent events에 `CPU_RECOVERED`
- Host 상태가 빨강에서 정상으로 변경

즉 Controller/UI는 "장애 넣는 화면"에서

장애-복구 시나리오를 완결시키는 실험 콘솔이 된다.

프로세스별 변화 크기 요약

변화 큼

- Host Simulator
- Local Agent
- Monitor
- Controller / UI

변화 작음

- Relay R1
- Relay R2

이유는 간단하다.

- 복구는 상태 전이와 해석 문제
- Relay는 그 이벤트를 전달만 하면 됨

가장 실용적인 추가 항목만 뽑으면

각 프로세스에 최소 추가할 건 이 정도다.

Host Simulator

- 장애별 정상 복귀 로직
- recover 명령 명확화

Local Agent

- recovery 이벤트 타입 추가
- 비정상 → 정상 전이 감지 규칙 추가

R1

- recovery 이벤트도 동일하게 forward

R2

- recovery 이벤트도 동일하게 retry/buffer/failover

Monitor

- recovery 이벤트 수신 시 상태 정상화
- active issue 제거

Controller/UI

- Recovery 버튼 세분화
- recent events / current status에서 recovery 반영

한 줄 결론

1~6 프로세스를 새로 갈아엎을 필요는 없고, “복구를 상태 전이 이벤트로 정식 취급”하도록 각 프로세스에 최소 보강을 하면 된다.

실제로 중요한 변경은 **Host Simulator, Local Agent, Monitor, Controller/UI**에 있고, **R1/R2**는 **recovery** 이벤트를 일반 이벤트처럼 전달하면 거의 충분하다.